

Introduction to neural networks

Gradient descent introduction

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

August 26, 2026©



Contents

- 1 Motivation
- 2 Gradient
- 3 Gradient descent

Roadmap

- 1 Motivation
- 2 Gradient
- 3 Gradient descent

From Rosenblatt's Rule to Gradient Descent

Rosenblatt's Perceptron Rule (1958)

- Works only for linearly separable problems.
- Provides no meaningful update when classes are not separable.



Figure: $w = w + \eta(y - \hat{y})$

From Rosenblatt's Rule to Gradient Descent (2)

Why Gradient Descent?

- Defines an explicit *loss function* $L(\mathbf{w})$ (e.g., squared error, cross-entropy).
- Updates are derived systematically:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L(\mathbf{w}_t).$$

- Extends naturally to multi-layer networks and differentiable activations.
- Provides convergence guarantees (under assumptions) and flexibility with optimizers.

Key insight: Gradient descent generalizes Rosenblatt's idea, moving from “correction by mistake” to “optimization by minimizing a loss.”

Roadmap

- 1 Motivation
- 2 Gradient
- 3 Gradient descent

- If f is a function of two variables x and y , then the gradient of f is the vector function ∇f defined by

$$\nabla f(x, y) = [f_x(x, y), f_y(x, y)] = \frac{\partial f}{\partial x}i + \frac{\partial f}{\partial y}j \quad (1)$$

[Stewart, Multivariate Calculus]

Gradient

- It is a vector valued function.

$$y = f(w_1, w_2, \dots, w_n)$$

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \dots \\ \frac{\partial f}{\partial w_n} \end{bmatrix}$$



- Example

$$f(x, y) = \begin{bmatrix} xy \\ xy \end{bmatrix}$$

$$\nabla f = ?$$

- Example

$$f(x, y) = \begin{bmatrix} xy \\ xy \end{bmatrix}$$

$$\nabla f = ?$$

$$\nabla f = \begin{bmatrix} y \\ x \end{bmatrix}$$

- A nice explanation: khanacademy.org/.../gradient

Summary: Fundamental Differentiation Rules

Basic Rules

- **Constant:** $\frac{d}{dx}(c) = 0$
- **Power:** $\frac{d}{dx}(x^n) = nx^{n-1}$
- **Constant Multiple:** $\frac{d}{dx}[cf(x)] = cf'(x)$
- **Sum/Difference:** $(f \pm g)' = f' \pm g'$

Operations & Composition

- **Product Rule:**

$$(fg)' = f'g + fg'$$

- **Quotient Rule:**

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$$

- **Chain Rule:**

$$[f(g(x))]' = f'(g(x)) \cdot g'(x)$$

Roadmap

- 1 Motivation
- 2 Gradient
- 3 Gradient descent**

- Goal. Minimize a differentiable objective

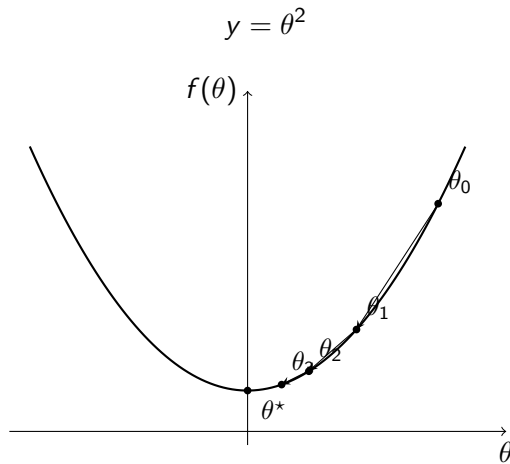
$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} f(\boldsymbol{\theta}), \quad \nabla f(\boldsymbol{\theta}) \text{ available.}$$

- Descent direction. The gradient points to steepest *increase*. Move in the opposite direction:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla f(\boldsymbol{\theta}_t),$$

where $\eta > 0$ is the *learning rate* / *step size*.

Geometric intuition.



Basic algorithm

- 1 Choose initial point θ_0 and step size $\eta \in (0, 1)$.
- 2 For $t = 0, 1, 2, \dots$ until convergence:
 - 1 Compute gradient $\mathbf{g}_t = \nabla f(\theta_t)$.
 - 2 Update $\theta_{t+1} = \theta_t - \eta \mathbf{g}_t$.

Choosing the step size

- Fixed η : simple; may under/overshoot.
- Schedules: $\eta_t = \eta_0 / (1 + \alpha t)$ or cosine decay, etc.
- Line search: adapt η_t each step to satisfy sufficient decrease.

Theory snapshots

- If f is L -smooth and μ -strongly convex, GD with $0 < \eta < 2/L$ converges linearly to θ^* :

$$\|\theta_t - \theta^*\| \leq (1 - \mu\eta)^t \|\theta_0 - \theta^*\|.$$

- For general non-convex f , GD decreases f and often converges to a *stationary point* where $\|\nabla f\|$ is small.

- Goodfellow, Ian, et al. Deep learning. Vol. 1. No. 2. Cambridge: MIT press, 2016.
- Stewart, James, Daniel K. Clegg, and Saleem Watson. Calculus: early transcendentals. Cengage Learning, 2020.
- Khan Academy, Calculus.